

Documento de entrega

Examen Final Semana 10 - API REST para gestion de clientes y pedidos

Nombre del estudiante	Damian Cespedes
Nombre del modulo	Taller de plataformas web
Nombre del docente	Zugehy Escalante Issele
Semana	10
Fecha	22-05-2026
Repositorio publico GitHub	PENDIENTE_DE_REEMPLAZAR_POR_EL_LINK_DE_TU_REPOSITORIO

Nota importante antes de entregar: reemplazar el enlace pendiente por el link real del repositorio publico de GitHub.

1. Introduccion

El presente trabajo desarrolla una solucion backend mediante una API REST conectada a una base de datos relacional. La solucion permite gestionar clientes y pedidos, aplicando operaciones CRUD, validaciones de datos, separacion de rutas y controladores, uso de variables de entorno y codigos de estado HTTP adecuados.

El objetivo principal es integrar los aprendizajes de desarrollo backend, bases de datos y APIs REST, creando una aplicacion funcional, ordenada y verificable mediante pruebas.

2. Desarrollo

La solucion fue desarrollada con Node.js, Express y SQLite. Se implemento una base de datos relacional compuesta por dos entidades principales: clientes y pedidos.

Entidad clientes:

- id_cliente: clave primaria autoincremental.
- nombre: nombre del cliente.
- email: correo electronico unico del cliente.
- telefono: telefono de contacto.
- created_at y updated_at: fechas de control.

Entidad pedidos:

- id_pedido: clave primaria autoincremental.
- id_cliente: clave foranea relacionada con clientes.
- cliente: nombre del cliente asociado.
- email_cliente: correo del cliente asociado.
- fecha_pedido: fecha del pedido.
- total: monto total del pedido, validado para ser mayor a 0.
- estado: estado del pedido.

La relacion entre las tablas se implemento mediante una llave foranea desde pedidos.id_cliente hacia clientes.id_cliente. Esta relacion permite que cada pedido quede asociado a un cliente preexistente y mantiene integridad referencial en la base de datos.

Endpoints implementados

Metodo	Ruta	Descripcion
GET	/api/clientes	Listar clientes
GET	/api/clientes/:id	Obtener cliente por ID
GET	/api/clientes/:id/pedidos	Listar pedidos de un cliente
POST	/api/clientes	Crear cliente
PUT	/api/clientes/:id	Actualizar cliente
DELETE	/api/clientes/:id	Eliminar cliente
GET	/api/pedidos	Listar pedidos
GET	/api/pedidos?estado=pagado	Filtrar pedidos por estado
GET	/api/pedidos?id_cliente=1	Filtrar pedidos por cliente
GET	/api/pedidos/:id	Obtener pedido por ID
POST	/api/pedidos	Crear pedido asociado a cliente existente
PUT	/api/pedidos/:id	Actualizar pedido
DELETE	/api/pedidos/:id	Eliminar pedido

Codigos HTTP usados

- 200 OK para consultas y actualizaciones correctas.
- 201 Created para creacion de clientes y pedidos.
- 204 No Content para eliminacion exitosa.
- 400 Bad Request para datos invalidos.
- 404 Not Found para recursos inexistentes.
- 409 Conflict para conflictos de integridad, como email duplicado o cliente con pedidos asociados.

3. Pruebas realizadas

Se realizaron pruebas de funcionamiento para verificar el correcto comportamiento de la API. El proyecto incluye pruebas para REST Client, coleccion de Postman y un script automatizado con npm run test-api.

Prueba	Resultado esperado
Crear cliente	HTTP 201
Listar clientes	HTTP 200
Actualizar cliente	HTTP 200
Crear pedido asociado a cliente existente	HTTP 201
Actualizar pedido	HTTP 200
Validar total menor o igual a 0	HTTP 400
Validar cliente inexistente	HTTP 404
Eliminar cliente con pedidos asociados	HTTP 409
Eliminar pedido	HTTP 204
Eliminar cliente sin pedidos asociados	HTTP 204

Las pruebas quedaron documentadas en la carpeta requests y en el archivo docs/PRUEBAS_FUNCIONAMIENTO.md.

4. Conclusion

El desarrollo de esta API REST permitio aplicar de forma practica los contenidos relacionados con backend, base de datos relacional y operaciones CRUD. La implementacion de una relacion entre clientes y pedidos mediante llave foranea permite mantener integridad entre las entidades y representar un caso real de gestion de pedidos.

Como proyeccion, esta solucion podria ampliarse agregando autenticacion, manejo avanzado de productos, detalle de pedidos y despliegue en un servidor web. Esto permitiria transformar la API en una base para un sistema mas completo de ventas o administracion comercial.